Reporte del Trabajo Práctico N.º 2

Programación I - Controlador de misiones para robot (UadeRobotLab)

Dato	Detalle
Universidad / Materia	UADE - Programación I
Docentes	Prof. G. Guzmán - Prof. N. Rosio
Integrantes	Juan Ignacio Chedufau, [Integrante 2], [Integrante 3], [Integrante 4]
Repositorio	[enlace al fork grupal de UadeRobotLab]
Archivo desarrollado	mi_desarrollo/mi_tp02.py
Fecha de entrega	02/11/2026

1. Objetivo del trabajo

El trabajo consistió en completar un controlador de misiones para el robot del laboratorio: un programa que recibe una lista de órdenes, valida cada una, ejecuta las correctas, rechaza las incorrectas sin interrumpir la misión y, al final, muestra un reporte con el resultado. Para eso se completaron las cuatro funciones indicadas en el docstring de *mi_tp02.py*: **comando_es_valido**, **ejecutar_comando**, **ejecutar_mision** y **generar_reporte**. El desarrollo se verificó en el simulador antes de la prueba con el robot real.

2. Modalidad de desarrollo

Modalidad híbrida: programación manual asistida por Inteligencia Artificial. El grupo dividió el trabajo en cuatro partes, una por integrante, que coinciden con las cuatro funciones a implementar. Cada integrante usó la IA como apoyo para entender la especificación, proponer una primera versión y resolver dudas puntuales, y luego revisó, ajustó e integró el código de forma manual. Las **pruebas fueron 100 % manuales**, ejecutando las misiones en el simulador y comparando el comportamiento del robot y el reporte final con lo esperado.

Parte	Función	Responsabilidad
1	comando_es_valido(comando)	Validar nombre del comando, cantidad y tipo de parámetros, y respetar los límites (0,20 m/s de avance, 0,50 rad/s de giro, 10 s por orden).
2	ejecutar_comando(robot, comando)	Traducir cada orden válida a la llamada correspondiente del robot (avanzar, girar, detenerse, saludar).
3	ejecutar_mision(robot, mision, historial)	Recorrer la misión completa, rechazar los comandos inválidos sin cortar la ejecución y capturar ErrorDeSeguridad con try/except, registrando todo en el historial.
4	generar_reporte(historial)	Mostrar el resumen final: comandos ejecutados, rechazados y el detalle de cada uno.

3. Tiempo empleado

Se informa la duración de cada actividad (tiempo de reloj) y el esfuerzo total en minutos-persona, ya que algunas tareas se hicieron en grupo y otras en paralelo.

Actividad	Modalidad	Duración (min)	Esfuerzo (min-persona)
Análisis	Reunión grupal (4 integrantes)	60	240
Programación	Individual, 1 parte por integrante, en paralelo	60	240
Pruebas	Grupal, con el código integrado, en el simulador	120	480
Presentación (incluye reporte)	Redacción y revisión del informe y del repositorio	30	30
TOTAL		**270**	**990**

4. Pruebas realizadas

Con las cuatro funciones integradas, el grupo ejecutó en el simulador las cuatro misiones provistas por la cátedra en *misiones.py*:

Misión	Qué se verificó	Resultado
MISION_BASICA	Ejecución completa de una misión sin errores.	OK
MISION_CUADRADO	Secuencia de avances y giros que forman un recorrido cuadrado.	OK
MISION_CON_ERRORES	Rechazo de comandos inválidos sin cortar la misión: tiempo excesivo, sentido de giro no numérico, comando inexistente ("volar"), velocidad no numérica y velocidad mayor al límite. El resto de la misión debe continuar.	OK
MISION_LARGA	Comportamiento con muchas órdenes seguidas y reporte final correcto.	OK

5. Uso de Inteligencia Artificial

Aspecto	Detalle
Herramienta	Claude (Anthropic)
Modelo	Claude Opus 5.5 - usado por los cuatro integrantes
Modalidad de uso	Asistente conversacional para análisis de la consigna, generación de una primera versión de cada función y resolución de dudas; el código final se revisó e integró manualmente.
Uso en pruebas	Ninguno: el testing fue manual.

5.1 Tokens y gasto estimado

Las herramientas no informan los tokens consumidos en el uso conversacional, por lo que el valor es una **estimación**. Se consideró que cada sesión incluye la lectura del archivo y la consigna, varias iteraciones de preguntas y respuestas, y que en cada mensaje el modelo vuelve a procesar el contexto acumulado (por eso los tokens de entrada son muchos más que los de salida). La parte 3 se estimó algo más alta por su mayor complejidad (recorrido de la misión y manejo de excepciones).

Uso	Tokens de entrada	Tokens de salida	Costo estimado (USD)
Integrante 1 - comando_es_valido()	150.000	18.000	US$ 0.96
Integrante 2 - ejecutar_comando()	150.000	18.000	US$ 0.96
Integrante 3 - ejecutar_mision()	200.000	25.000	US$ 1.30
Integrante 4 - generar_reporte()	150.000	18.000	US$ 0.96
Redacción del reporte	100.000	15.000	US$ 0.70
TOTAL	**750.000**	**94.000**	**US$ 4.88**

Cálculo: precio de lista de la API de Anthropic para Claude Opus 5.5 = US$ 4.00 por millón de tokens de entrada y US$ 20.00 por millón de tokens de salida. Entrada: 0.750 M x US$ 4 = US$ 3.00. Salida: 0.094 M x US$ 20 = US$ 1.88. **Total estimado: aproximadamente US$ 4.88** (alrededor de 900.000 tokens en total).

Nota: el monto es el costo equivalente si se hubiera pagado por uso a través de la API. Si el modelo se usó dentro de una suscripción de Claude, ese consumo queda incluido en el abono y no genera un gasto adicional. Fuente del precio: página de precios de la plataforma de Claude (platform.claude.com/docs/en/about-claude/pricing), vigente a noviembre de 2026.

6. Conclusiones

Dividir el trabajo según las cuatro funciones permitió avanzar en paralelo, y la reunión inicial de análisis fue clave para que todos respetaran la misma estructura de datos (formato de los comandos y del historial) y las piezas encajaran al integrarlas. La IA aceleró la escritura de la primera versión, pero las pruebas grupales en el simulador fueron las que confirmaron el comportamiento correcto, sobre todo en la misión con errores, donde el controlador debe detectar los comandos inválidos y seguir con el resto en lugar de detenerse.